

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Implementation Roadmap & Traceability 6
 - 2.2 CLI Command Execution Workflow 6
 - 3. Detailed Technical Specifications & Business Rules 7
 - 3.1 Requirements Traceability 8
 - 3.2 Security Rules 10
 - 3.3 Data Models 10
 - 4. Project Governance & Structural Gaps 10
 - 4.1 Structural Gaps 10
 - 4.2 Remediation & Workflow 11
 - 5. Technical & Domain Glossary (Terminology Reference) 11

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The project is a Python-based CLI To-Do List Manager utilizing a JSON-based flat-file storage system for persistence. It implements a modular architecture separating the CLI entry point, service-layer logic, and data models. The system provides core task lifecycle management including creation, status updates, sequential ID assignment, and filtering via human-readable or JSON outputs.

1.2 Maturity Assessment

The specification is highly stable and structurally sound, reflecting a state of READY for execution. While there are medium-severity gaps regarding the formalization of Acceptance Criteria and low-severity omissions in security/performance constraints, the presence of explicit 'Independent Tests' for every user story provides sufficient functional coverage to proceed without further refinement.

1.3 Technical Stack

- Python
- pyproject.toml

1.4 Architectural Constraints

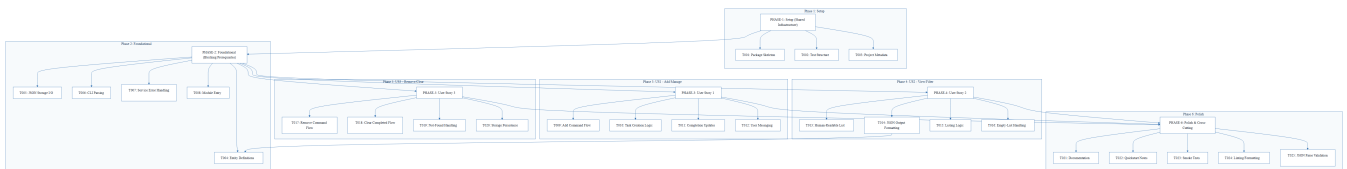
- **Strict sequential execution:** Phase 2 (Foundational) must be complete before any User Story implementation begins.
- **Storage pattern:** JSON file I/O for persistence located at `~/ . todos . json`.
- **Data integrity:** Sequential ID assignment and mandatory `created_at` timestamp population during task creation.
- **Output requirements:** Dual-mode listing (Human-readable and parseable JSON via `--json` flag).
- **Ordering constraint:** Preservation of task order and status fields during listing operations.
- **Error handling:** Mandatory not-found handling for invalid task IDs.

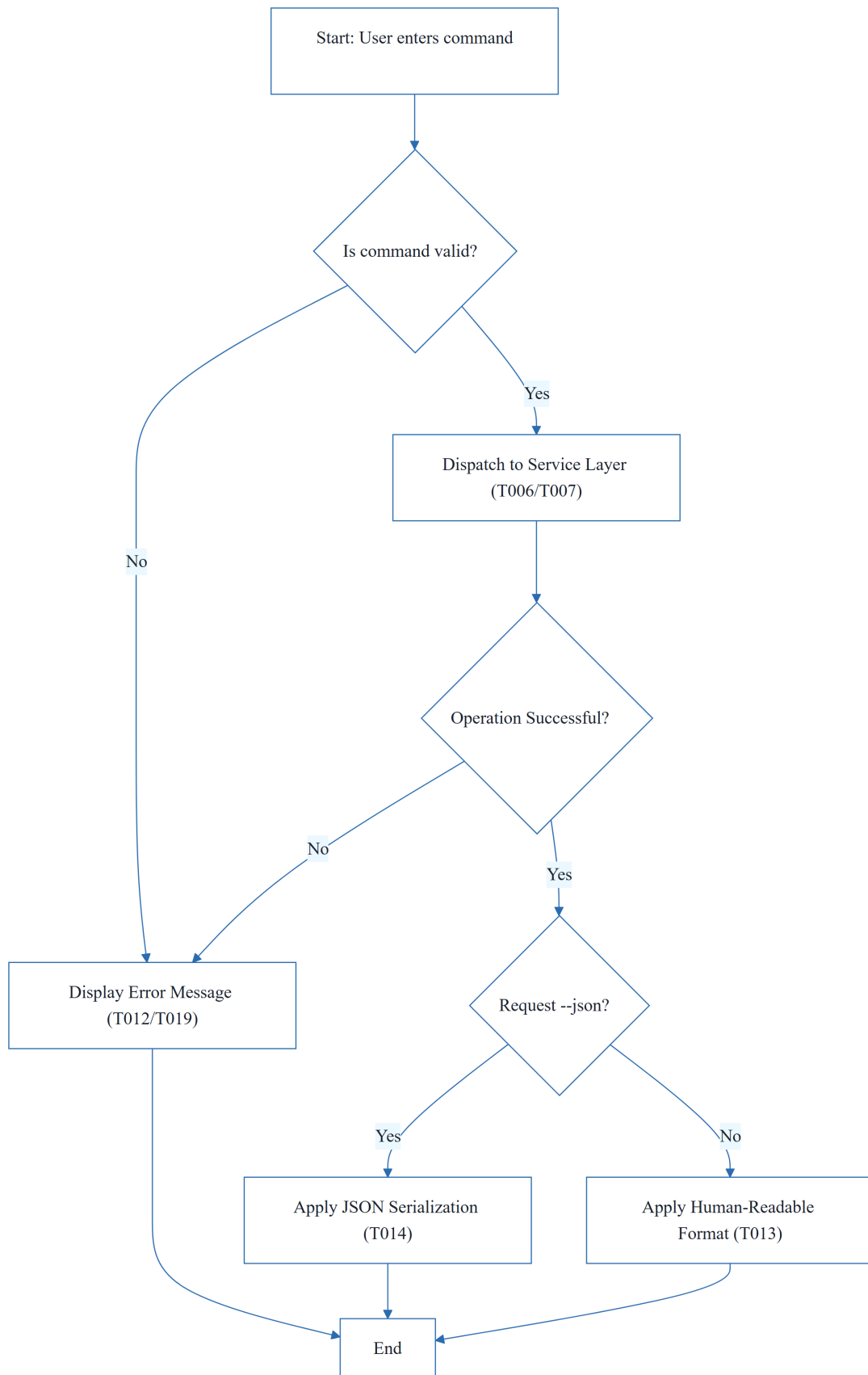
1.5 Critical Dependencies

- JSON storage path resolution and file I/O helpers in `src/todo_manager/storage.py`.
- Task entity definitions and serialization helpers in `src/todo_manager/models.py`.

- Service-layer error handling and task collection utilities in `src/todo_manager/service.py`.
- Referential integrity between CLI command flow and service-layer logic.
- Dependency of Phase 6 (Polish) on the completion of all preceding User Stories (US1, US2, US3).

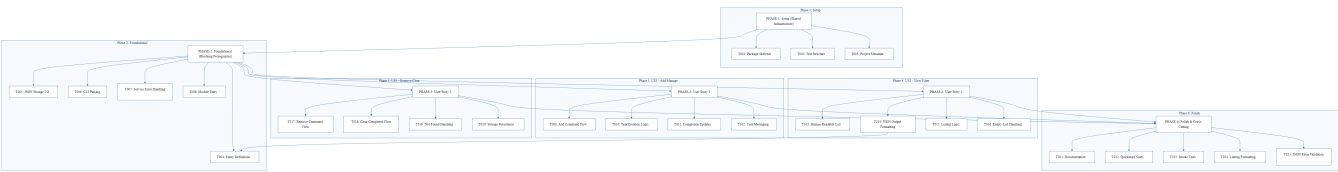
2. Architecture Workflows



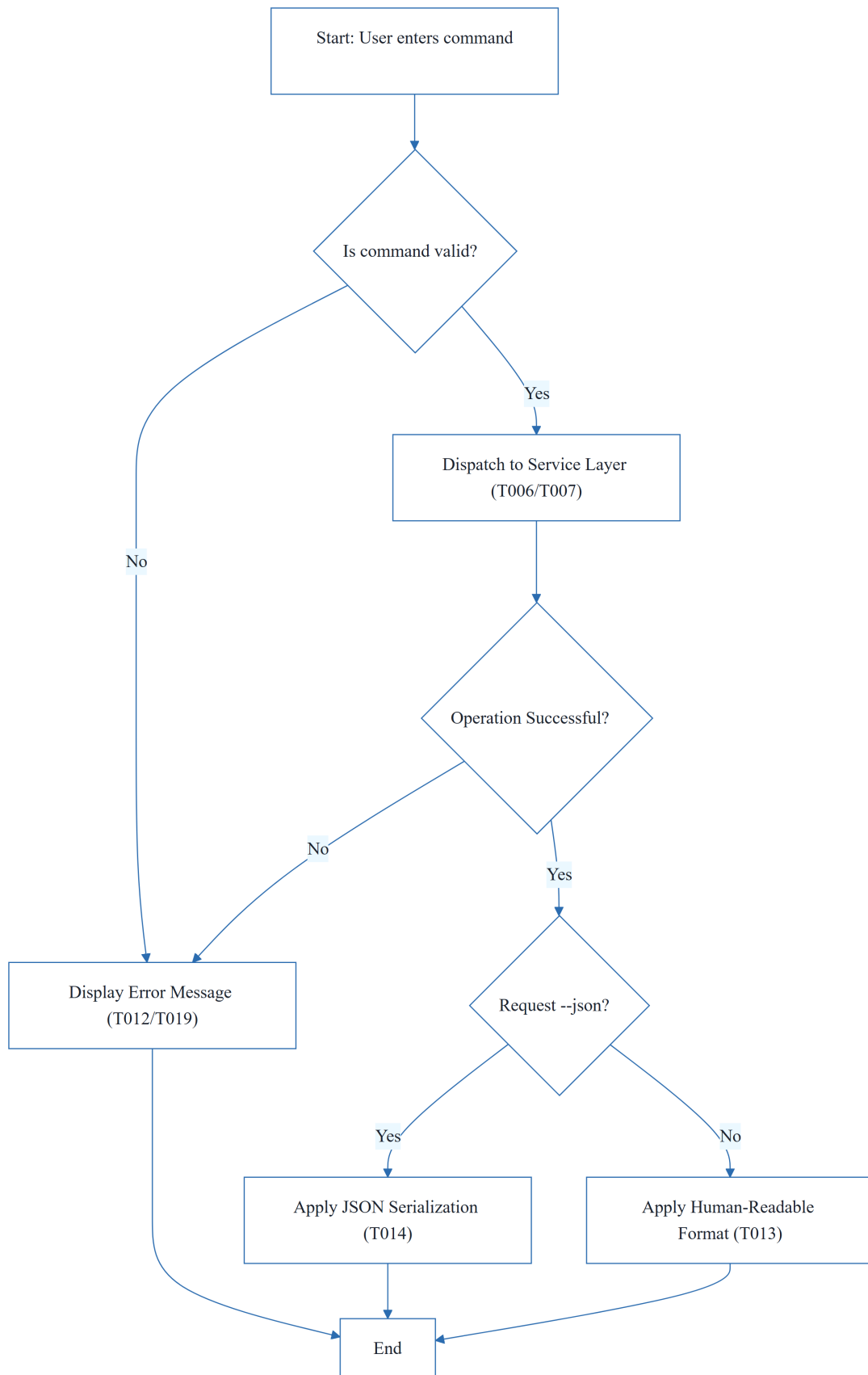


& Visual Diagrams

2.1 Project Implementation Roadmap & Traceability



2.2 CLI Command Execution Workflow



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Requirement / Task Description	Source Section	Status
T001	Create the Python package skeleton in <code>src/todo_manager/</code>	Phase 1: Setup	completed
T002	Create the test directory structure in <code>tests/unit/</code> and <code>tests/integration/</code>	Phase 1: Setup	completed
T003	Add project metadata and tooling entry points in <code>pyproject.toml</code>	Phase 1: Setup	completed
T004	Implement task entity definitions and serialization helpers in <code>src/todo_manager/models.py</code>	Phase 2: Foundational	completed
T005	Implement JSON storage path resolution and file I/O helpers in <code>src/todo_manager/storage.py</code>	Phase 2: Foundational	completed
T006	Implement shared command-line parsing and top-level dispatch in <code>src/todo_manager/cli.py</code>	Phase 2: Foundational	completed
T007	Implement shared service-layer error handling and task collection utilities in <code>src/todo_manager/service.py</code>	Phase 2: Foundational	completed
T008	Define module entry behavior for python <code>-m todomanager</code> in <code>src/todomanager/main.py</code>	Phase 2: Foundational	completed
T009	Implement the add command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	Implementation for User Story 1	completed
T010	Implement task creation, sequential ID assignment, and createdat <i>population</i> in <code>src/todomanager/models.py</code>	Implementation for User Story 1	completed
T011	Implement completion updates for existing tasks in <code>src/todo_manager/service.py</code>	Implementation for User Story 1	completed

ID	Requirement / Task Description	Source Section	Status
T012	Add user-facing success and error messages for add and complete operations in <code>src/todo_manager/cli.py</code>	Implementation for User Story 1	completed
T013	Implement human-readable task listing output in <code>src/todo_manager/cli.py</code>	Implementation for User Story 2	completed
T014	Implement <code>--json</code> output formatting in <code>src/todo_manager/cli.py</code>	Implementation for User Story 2	completed
T015	Add listing logic that preserves task order and status fields in <code>src/todo_manager/service.py</code>	Implementation for User Story 2	completed
T016	Handle the empty-list case with a clear message in <code>src/todo_manager/cli.py</code>	Implementation for User Story 2	completed
T017	Implement the remove command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	Implementation for User Story 3	pending
T018	Implement the clear command flow for removing completed tasks in <code>src/todo_manager/service.py</code>	Implementation for User Story 3	pending
T019	Add not-found handling for invalid task IDs in <code>src/todo_manager/cli.py</code>	Implementation for User Story 3	completed
T020	Ensure storage writes persist removals and clears safely in <code>src/todo_manager/storage.py</code>	Implementation for User Story 3	completed
T021	Document the CLI usage and storage behavior in <code>README.md</code>	Phase 6: Polish	pending
T022	Add quickstart verification notes and examples in <code>specs/001-cli-todo-manager/quickstart.md</code>	Phase 6: Polish	pending
T023	Run a manual smoke test of add, list, complete, remove, and clear against <code>~/todos.json</code>	Phase 6: Polish	pending

ID	Requirement / Task Description	Source Section	Status
T024	Verify linting and formatting expectations for the source files in <code>src/todo_manager/</code>	Phase 6: Polish	pending
T025	Confirm the generated JSON output remains parseable for the todo list <code>--json</code> path	Phase 6: Polish	pending
CRIT-FOUNDATION	No user story work can begin until Phase 2 is complete	Phase 2: Foundational	N/A

3.2 Security Rules

- **File Access:** JSON storage is located at `~/ .todos.json`. Access is governed by the operating system's user-level file permissions.
- **Input Validation:** Command-line arguments must be parsed and validated via the CLI dispatch layer (T006) before being passed to the service layer.

3.3 Data Models

- **Task Entity:** Defined in `src/todo_manager/models.py`.
- **Attributes:**
 - `id`: Unique sequential alphanumeric token.
 - `created_at`: Automatic timestamp population upon instantiation.
 - `status`: Completion state.
- **Persistence:** Flat-file JSON serialization.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	The document contains 'Independent Tests' for each story; these should be formalized into a dedicated Acceptance Criteria section.
Security & Performance Constraints	LOW	Define constraints for JSON file access permissions or maximum task list sizes.
Open Questions & Uncertainties	LOW	No uncertainties were listed in the task document.

4.2 Remediation & Workflow

The project follows an incremental delivery model:

1. **Setup & Foundational:** Establish the core infrastructure.
2. **User Story Implementation:** Execute US1 (MVP), then US2, then US3.
3. **Validation:** Each story is validated via its specific "Independent Test" before proceeding.
4. **Polish:** Final smoke tests and documentation.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A synchronization gate ensuring all prerequisites are satisfied before concurrent development of functional requirements commences.
Foundational	TECHNICAL_STACK	PHASE-2	The architectural base layer consisting of shared utilities, entity definitions, and core I/O logic that blocks all subsequent user story implementation.
Goal	BUSINESS_DOMAIN	PHASE-3	The specific operational outcome a user must achieve via the interface to satisfy a functional requirement.
ID	TECHNICAL_STACK	T010	A unique, sequential alphanumeric token assigned to each record for precise targeting and retrieval.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage and machine-readable output.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimum viable set of capabilities, specifically add and complete operations, required for the first functional validation.

Term	Category	Context Anchor	Project Definition
Organization	BUSINESS_DOMAIN	Tasks: CLI To-Do List Manager	The method of grouping development efforts by user story to ensure independent implementability and verification.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of required design and contract documents that must be analyzed before executing the implementation plan.
README	TECHNICAL_STACK	T021	The primary documentation file detailing interface usage and persistence behavior for end-users.
Setup	TECHNICAL_STACK	PHASE-1	The initial phase of creating the package skeleton, directory structures, and build configuration.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for providing a clear interface message when the task collection is empty.
Tests	TECHNICAL_STACK	T002	The validation framework comprising unit and integration directory structures for verifying system behavior.
population in	TECHNICAL_STACK	T010	The automatic assignment of timestamp values to the creation field during record instantiation.
todo clear	BUSINESS_DOMAIN	T018	The operation of purging all records marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operation of retrieving and displaying all records in either human-readable or structured formats.
■■ CRITICAL	TECHNICAL_STACK	CRIT-FOUNDATION	A strict dependency constraint prohibiting any functional development until the shared infrastructure phase is verified.